

One person, eight names.

A Graph RAG system over 190 unsealed federal court filings. Built on ArangoDB, measured earnestly, and brutally honest about where it falls short.

3,338 PASSAGES

19,009 MENTIONS

2,014 PEOPLE, PLACES, ORGS

488 SAME-PERSON LINKS

BACKGROUND · 6–8 MINUTES

My bio, why this role

SUMMARY & ENGAGEMENTS

Where you are now, in one sentence.

Two or three engagements — favour ones where you explained something technical to a non-technical decision maker.

GRAPH EXPERIENCE & WHY THIS ROLE

Which systems, at what depth, on what problems.

Why ArangoDB, why now, why solutions architecture.

The rest of this deck is one long answer to the same question: I didn't read about Graph RAG. I built one, measured it, and can tell you where it fails.

What the suite actually does

Two services. One draws the map, the other reads it.

- The Importer builds, the Retriever answers. Raw files go in one end and come out as a knowledge graph; questions go to the other.

In other words: you can redraw the map without touching the thing your users talk to.

- Four steps to build it. Split the documents into passages, have a model find the people, places and organisations, group related ones into topics, then have the model write a short report on each topic.

In other words: it reads everything once, so nobody on your team has to read it again.

- Every answer cites the page it came from.

In other words: a person can check the work — which is the difference between a tool a regulated business can use and one it cannot.

- It runs where the data already lives, including air-gapped, against models on your own hardware.

In other words: nothing leaves the building, and one database replaces three to secure and staff.

The executive question is never "how does the retriever work". It is what can my team ask on Monday that they could not ask on Friday — and can they trust the answer.

Five ways to ask a question

Different business questions need different retrieval. Most products ship one.

Method	Answers	Speed
Global	Themes across every document, using the topic reports	medium
Local	One person or organisation, and everything linked to it	low
Deep	Multi-step research; the model plans its own steps	higher
Instant	Fast answers with references	low
Custom	Your own rules, over your own data	varies

- Global Search is the one nobody else ships. It reads the pre-written topic reports instead of walking the graph, so it can answer a question about the collection as a whole.

In other words: "what is actually in here?" — the first thing anyone asks, and the one thing ordinary search cannot do.

- Graph search needs a name to start from. Ask "which courts appear in these documents" and there is nothing to anchor on. Mine falls back to plain similarity search on 11 of 35 test questions for exactly that reason.

In other words: this is where my design stops and theirs keeps going. I would rather show you the gap than hide it.

Ideal fit, and less than ideal fit

IDEAL FIT

- Relationships carry the meaning — investigations, compliance, supply chain.
- Answers span documents — two facts in two files must be joined.
- Questions about the whole set — counting and summarising, where similarity has nothing to rank.
- Data that cannot leave — regulated, privileged, classified.

NOT THE BEST FIT

- Purely conceptual questions. There is no name to start from, so plain meaning-based search (cosine) wins.
- Small, uniform document sets. One author and one vocabulary, so the extra machinery earns nothing.
- Simple lookup. If every question is "find the passage about X", a plain vector index is the answer.

The Importer has a setting, `rag_mode: "vector_rag"`, that skips entity extraction altogether. The product already agrees with this column.

Solutions comparison at a glance

Alternative	Its appeal	Where it stops helping
Vector DB alone	Simplest thing that works	No structure. It cannot answer "both X and Y", and it cannot count.
Neo4j + LangChain	Mature engine, large hiring pool	You assemble the RAG layer yourself. Vectors live in a second database, and the join between the two is where bugs live.
Microsoft GraphRAG	Where the idea was published	A library, not a service. You operate all of it.
Off-the-shelf frameworks	Fastest path to a demo	The demo is the easy part. Matching the same person across documents, and re-loading changed documents, are where the year goes.

What I won't claim: I haven't benchmarked ArangoDB against Neo4j or Neptune. ArangoDB is written in C++ rather than Java, so there are no garbage-collection pauses and the memory floor is lower. My 30–91 ms graph queries are consistent with that. They are not proof of it.

What ArangoDB excels at — and where mine might complement

The kind of documents you have decides which approach works.

WHAT ARANGODB EXCELS AT

- Groups the graph into topics, and writes a report on each.
In other words: it can describe a whole collection, not just find a passage in it.
- Those reports absorb duplicate names on their own.
In other words: on well-named documents, the problem mostly solves itself.
- Best on precise material — documentation, reports, structured filings.
In other words: most business content is written this way.

WHERE MINE MIGHT COMPLEMENT

- That absorption stops at Local Search. Two nodes for one person, half the connections, no warning.
In other words: the answer looks complete when it is not.
- My documents name people loosely. RE CAREY, JOSEPH · Det. Recarey · Mr. Recarey — one man, one docket.
In other words: documents that name things consistently never need this step.

19,009 MENTIONS

2,014 ACTUAL PEOPLE

19.8h THE MOST EXPENSIVE STAGE

Said carefully: I read their published documentation, not their source, and I did not test the product. Their docs describe no identity-resolution step, but that is an argument from silence. The fair question is on their ground, not mine: how does the Importer handle k8s versus Kubernetes?

190 unsealed filings from a single civil docket, public record. Every model ran on my own machine; nothing was sent to an outside service.

Demonstration

01 A question about a person — watch the graph path. 1,770 passages reachable, eight returned.

02 A question the documents cannot answer — it reports that it found no name to start from, then declines: "no judicial opinions, only party filings."

03 The graph itself — Joseph Recarey's seven different spellings. Then Jane Doe 2, and fifteen matches the system refused to make.

04 AQL, ArangoDB's query language — the name family, the review queue, and how much more evidence a family reaches than a single spelling.

Watch the channel tags: vector#26 lexical#7 graph#11 means three independent methods each picked the same passage. When they agree, that is the strongest signal the system produces.

What I'd change to run this at scale

What I built proves the idea. It is not what I would deploy.

- I would delete the second database. The graph is in ArangoDB and the text and vectors are in LanceDB. ArangoDB 3.12 has a native vector index, so the two collapse into one.

In other words: a whole class of bug — the two stores quietly disagreeing — stops existing, and a join across two systems becomes one lookup.

- I would shard the graph by legal matter. A SmartGraph is split on an attribute you nominate; put every vertex of one matter on the same shard and a traversal never crosses the network. It is also the customer boundary.

In other words: one client's documents stay in one place — faster to query, and far easier to explain to their auditor.

- I would keep the entity index on every node as a satellite collection, because every single traversal touches it.

In other words: don't make the most-used lookup in the system take a network hop.

- I would budget and staff a human review step. 413 uncertain name matches are queued and nobody has looked at them. A wrong merge invents a connection between real people; a missed one only means we find less.

In other words: I would rather miss a link than invent one — and someone has to be paid to make that call.

- Unsolved: adding documents without rebuilding. Matching names across all 190 filings took 19 hours, and that cost grows with the number of mentions, not the number of documents.

In other words: today, one new filing means redoing the whole graph. That is the hard problem, and I have not solved it.

A question for you, not a criticism: the Technical Overview says SmartGraph creation through the Importer currently accepts only `shard_count: 1`. So is the sharding above something I would have to build outside the Importer today?

Pitfalls learned along the way

Each one taught me something I now carry into the next build and related projects.

1 · A LIBRARY DEFAULT COST 9.6% OF THE GRAPH

The name-matching library (Jaro-Winkler) defaults to case-sensitive. In documents full of ALL-CAPS captions, that produced 38 separate "Mr. Barton" nodes.

→ Fixed — compare both names in the same case, and sort the candidate list by score before cutting it to five. Then re-ran the pass over every document.

2 · TWO DIFFERENT MAXWELLS MERGED INTO ONE

Ghislaine and her father Robert. Comparing two names at a time cannot tell a fuller version of one name from a different person entirely.

→ Fixed — judge the whole family of names at once instead of pair by pair, using how often each appears, how widely across documents, and in what role. Anything still uncertain is flagged, not merged.

3 · THE MODEL NEVER SAW THE TOP OF ITS OWN INSTRUCTIONS

Ollama silently cuts any prompt down to 4,096 tokens, without warning — and it cuts from the front. Mine were 4,235, so every test run lost the opening of its own system prompt, starting with "use only these sources".

→ Fixed — set the context length explicitly to 8,192 in config rather than inheriting a default, and log the prompt length so the next cut is visible.

4 · BIGGER MODEL, WORSE RESULTS

A 30-billion-parameter model of the mixture-of-experts kind scored 11/16. A plain 8-billion model scored 15/16, at the same speed.

→ Settled — kept the smaller model, on the evidence. I also abandoned a prompt change that made both models worse, rather than trying to rescue it.

Every one was found by auditing real output — not by reading code, and not by a test. Every one is fixed and re-measured.

A human stayed in the middle of every step. Claude Code wrote most of the code. Every data structure, threshold and algorithm choice was argued out before it went in. Working the problem with the model rather than delegating it to the model is where the experience compounded.

Four channels, fused by rank

Channel	Mechanism	The question that justified it
vector	Meaning-based similarity (cosine)	One passage names him, and it ranks first by similarity
density	Documents with the most passages in the shortlist	A 102-passage transcript: ranked 14th by its best passage, 2nd by density
lexical	Keyword search after the model widens the query (BM25)	The word "obiter" appears nowhere; 35 passages say "held that"
graph	Match a name, follow one link, collect its mentions	"Both Maxwell and Epstein" — a single vector cannot express "both"

RANK FUSION (RRF)

$\Sigma 1/(60 + \text{rank})$. It uses position only, so channels that score on completely different scales can be combined without anyone inventing weights.

WHY HALF THE SLOTS ARE RESERVED

Half always go to plain similarity search. Treating all four channels equally once fixed one question and broke another in the same change.

Saying "I don't know" — two bugs, pulling opposite ways

Fixing the first one caused the second. Only measurement found it.

FIRST: IT ANSWERED EVERYTHING

Asked something the documents could not answer, it described what they did contain and offered that as the answer. Four prompt rewrites failed — the prompt already forbade it.

The cause was structural: one model, in one pass, both decided whether it had enough evidence and wrote the answer. Writing something is the easier path.

→ Fixed — split the two. Step 1 decides, and its reply format has no field big enough to hold prose. Step 2 writes, and runs only if step 1 said yes.

THEN: IT REFUSED THINGS IT COULD ANSWER

"Who were the defense attorneys in this case?" was declined. The answer was sitting in the top result: "MS. BORJA: Mary Borja for Defendant, Alan Dershowitz."

Step 1 read all eight passages in one call and gave a yes/no for each. That looks independent. It is not — remove one unrelated passage and the verdict on the top result flipped to yes. Swap in a different one and it flipped back.

→ Fixed — one passage per call, each call seeing exactly one passage. The other seven can no longer affect the verdict on the first.

0 / 15 REFUSED BEFORE

11 / 15 REFUSED AFTER

0 WRONG REFUSALS IN 105 RUNS

49s DOWN FROM 63S

What is measured, and what is not

MEASURED

35 questions in seven categories. The predicted winner was written down and locked before any strategy ran.

- Graph found no name to start from on 11 of 35
- Won at least one of the eight final slots on 18 of 35
- Changed the document set on 14 of 35

NOT MEASURED — DELIBERATELY

No human-marked correct answers, so none of the standard ranking scores (recall@k, nDCG) can be computed. Judging correctness requires a person, and cannot be delegated.

An answer key written by a model would measure the judge, not the systems.

The evaluation shows that the strategies behave differently. It does not show which one is right. I would rather say that than quote a number I cannot defend.

Challenges for improving the system

Each one is scoped, understood and costed. None of them is a surprise, which is the point of measuring a system rather than only building it.

- Same-person links for places and organisations. Limited to people for now; the matching rule has to change depending on the type.
- 413 flagged pairs awaiting a decision. The system knows what it is unsure about. It needs a screen that shows a reviewer.
- Correctness not yet verified by a person. Human work by design, not a gap waiting on automation.
- Answering a different question than the one asked. Separate from declining to answer, and not fixed by it.
- Adding new documents without rebuilding. The hardest, and the one that matters most at scale.